Task-1：基于机器学习的文本分类

一、实验目的

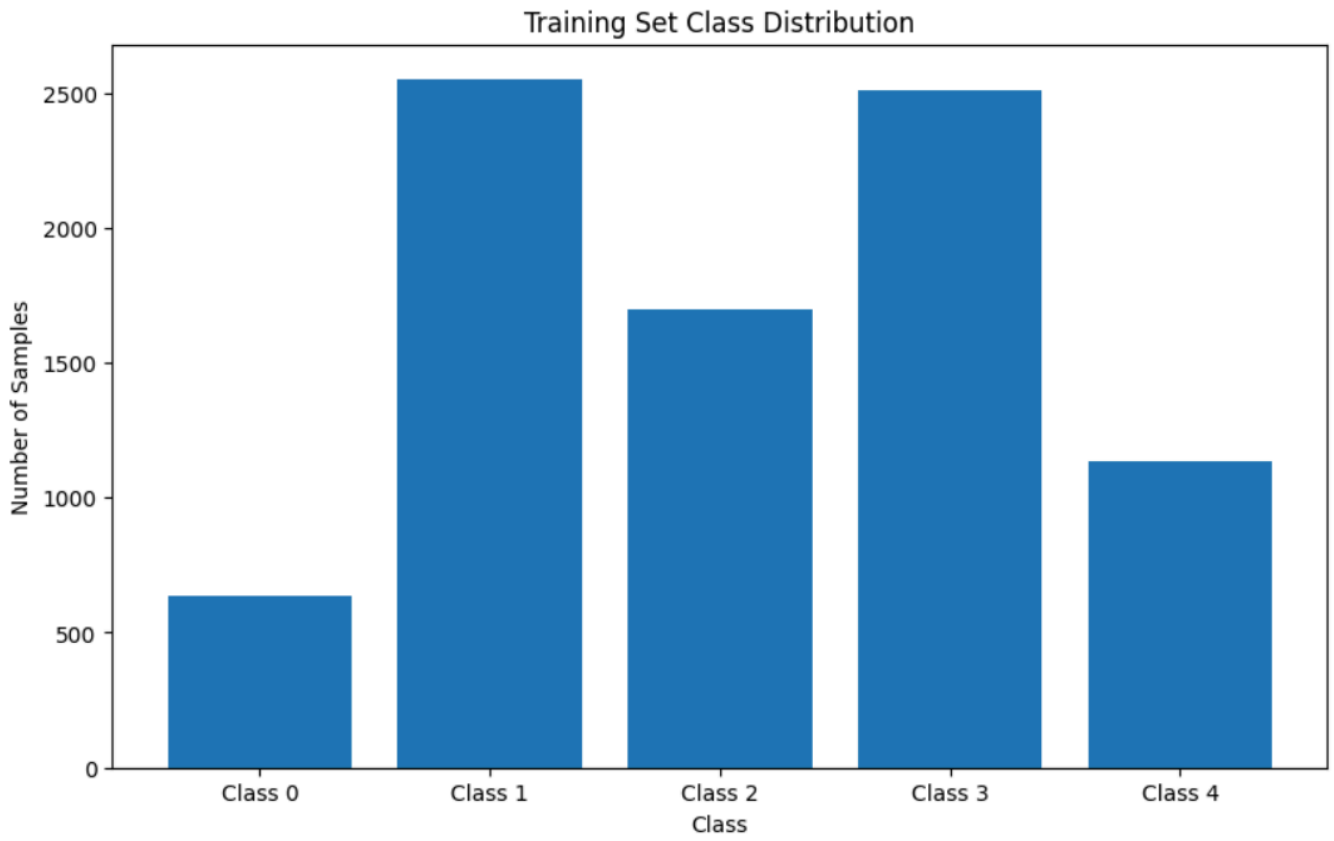
通过实现 Softmax 分类，掌握文本分类的基本流程，并分析不同词向量化超参数、损失函数和学习率对模型训练的影响。最后，针对模型准确率较低的问题，进行了进一步的实验与改进，并分析导致准确率较低的原因。

二、实验环境

- **平台：** Kaggle Notebook（GPU T4 $\times$ 2）
- **语言：** Python 3.10
- **核心工具包：** scikit-learn, PyTorch, numpy, pandas

三、实验数据

数据来源于飞书文档中的 `new_train.tsv` 与 `new_test.tsv`，共有 `8528` 条训练数据，`3309` 条测试数据，其类型数量分布如下图：



四、方法与步骤

1. 数据预处理

- **数据划分**：按照 `8:2` 的比例划分训练集与验证集。
- **文本向量化**：使用 `scikit-learn` 的 `CountVectorizer` 进行 `n-gram` 特征提取。当 `n = (1, 1)` 时退化为词袋模型（Bag-of-Words）。
- **特征限制**：设置 `max_features` 限制词汇表大小，避免维度爆炸。

2. 模型实现

- **模型架构**：实现了基于 **Softmax 回归** 的线性分类器，包含可学习的权重矩阵 `W` 和偏置向量 `b`。
- **训练策略**：
  - 采用 **mini-batch 梯度下降** 进行优化，批量大小为 `64`。
  - 实现 **早停机制**（`patience=10`），当验证集损失连续 `10` 轮未下降时停止训练，防止过拟合。
- **损失函数**：支持交叉熵（cross-entropy）、均方误差（MSE）、合页损失（hinge）和感知器损失（perceptron）四种选择。

3. 超参数调优

- **调优策略**：采用 **坐标下降法（Coordinate Descent）** 进行超参数搜索：
  1. 初始化一组默认超参数
  2. **固定其他参数**，每次只优化一个超参数
  3. 在**验证集**上评估效果，选择最佳值
  4. 固定该参数，继续优化下一个参数
- **调优参数**：
  - 特征提取：`n-gram` 范围 `n ∈ {(1,1), (1,2), (1,3), (2,2), (2,3)}`
  - 特征数量：`max_features ∈ {1000, 5000, 10000, 20000}`
  - 学习率：`learning_rate ∈ {0.001, 0.01, 0.1, 0.5}`
  - 损失函数：`loss ∈ {cross_entropy, mse, hinge, perceptron}`
- **最优参数**：最终确定最优组合为 `learning_rate = 0.01`，`n = (1, 2)`，`max_features = 10000`，损失函数为交叉熵。

五、实验与分析

1. 实验一：对比不同向量化方法

实验内容

- `n-gram` 参数选择：`(1,1), (2,2), (3,3), (1,2), (1,3), (2,3)`
- `max_features` 参数选择：`1000, 5000, 10000, 20000`

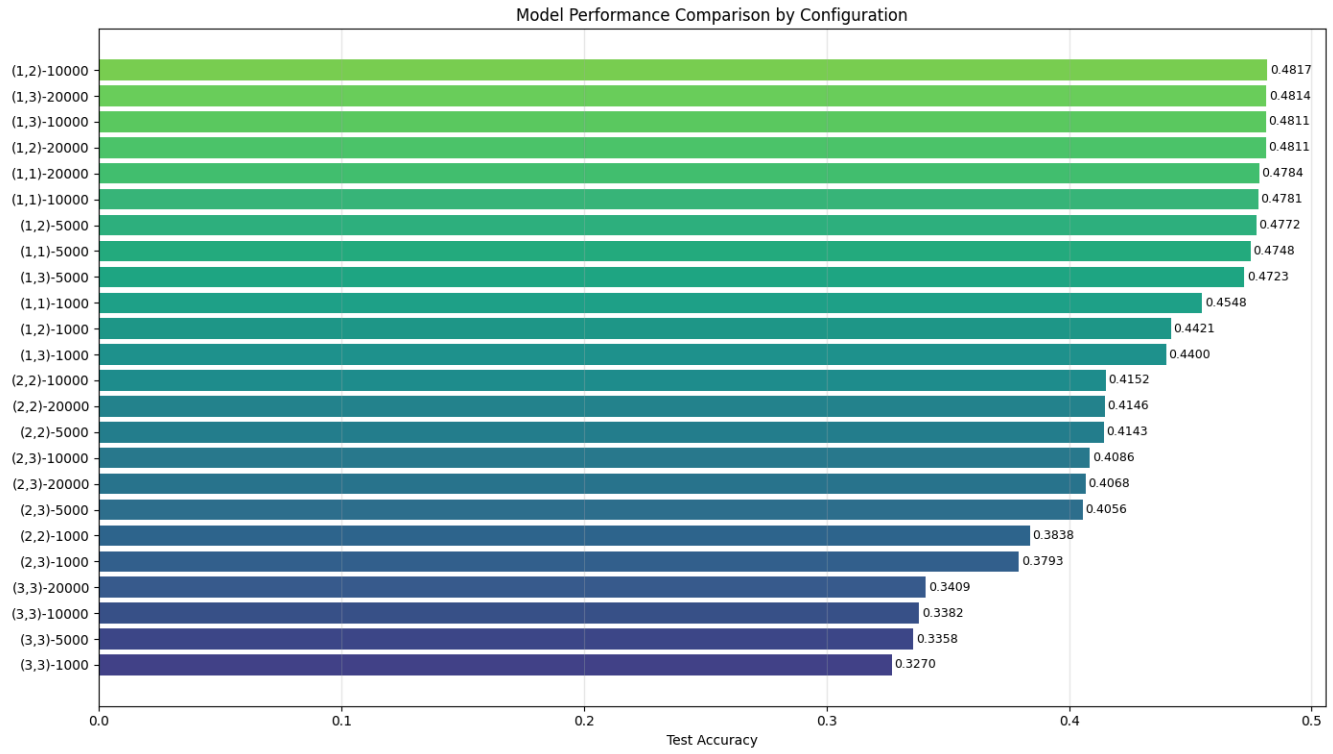
评估指标

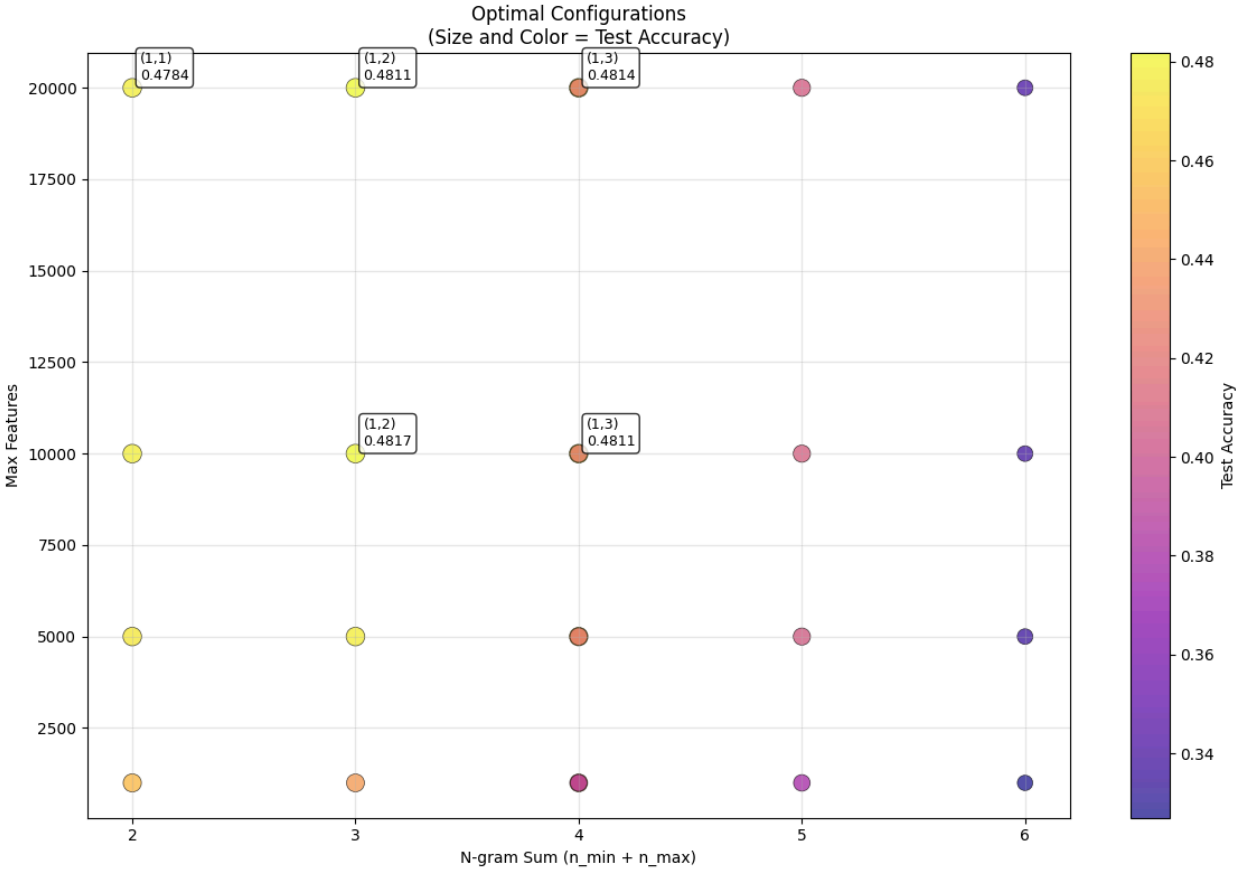
- 准确率；
- 特征维度；
- 训练时间；

实验结果

不同参数组合的准确率

不同参数下，模型在测试集上的准确率如下图所示：

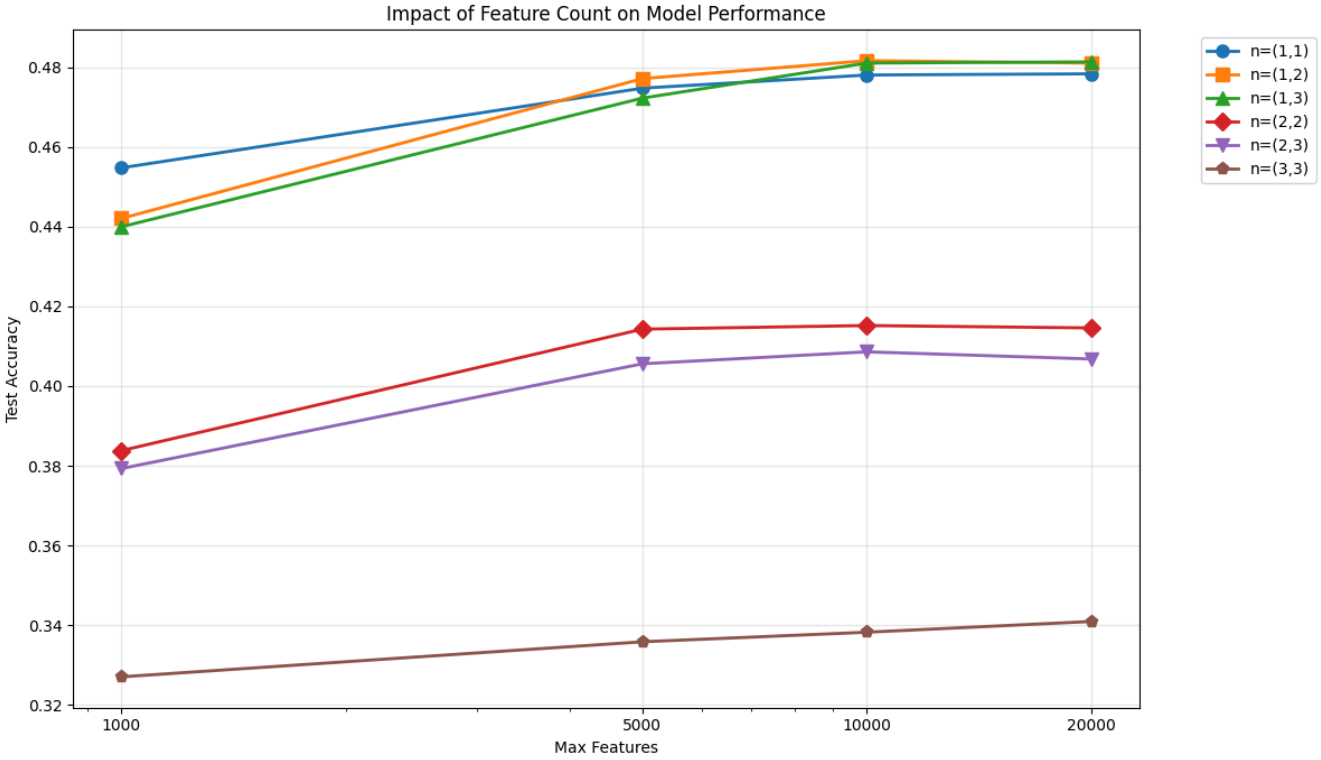


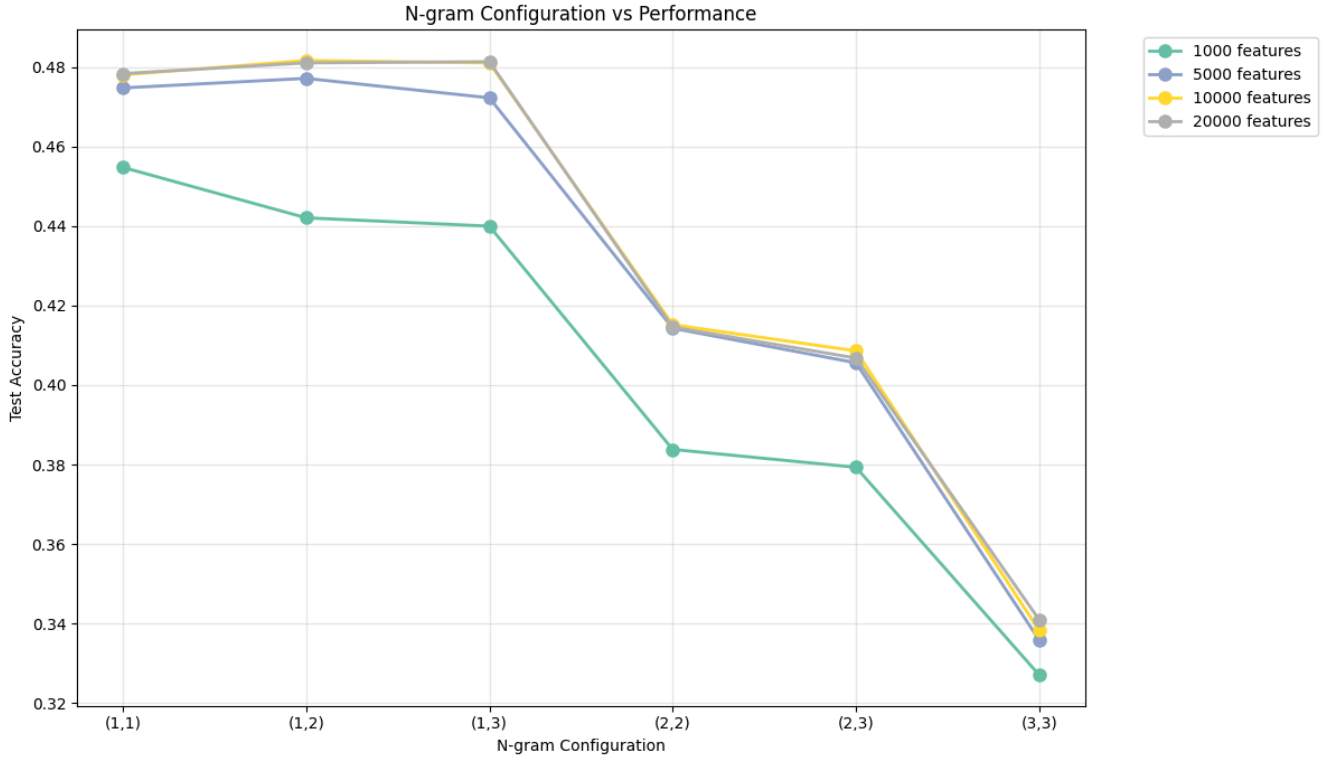


可知，最优参数（综合考虑训练时间和准确率）为 `n=(1,2), max_feature=10000`。

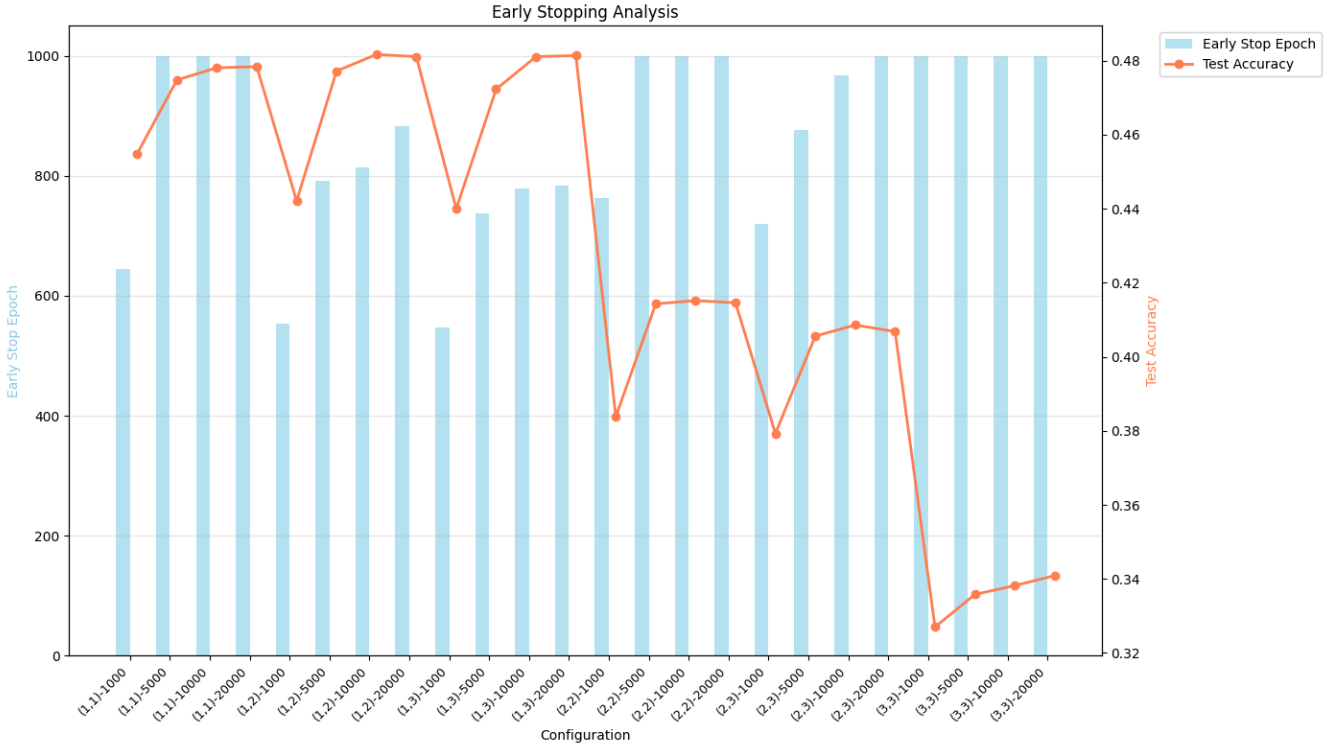
向量化参数对准确率的影响

`max_feature` 和 `n-gram` 参数对模型在测试集上的准确率的影响如下图所示：





向量化参数对早停的影响



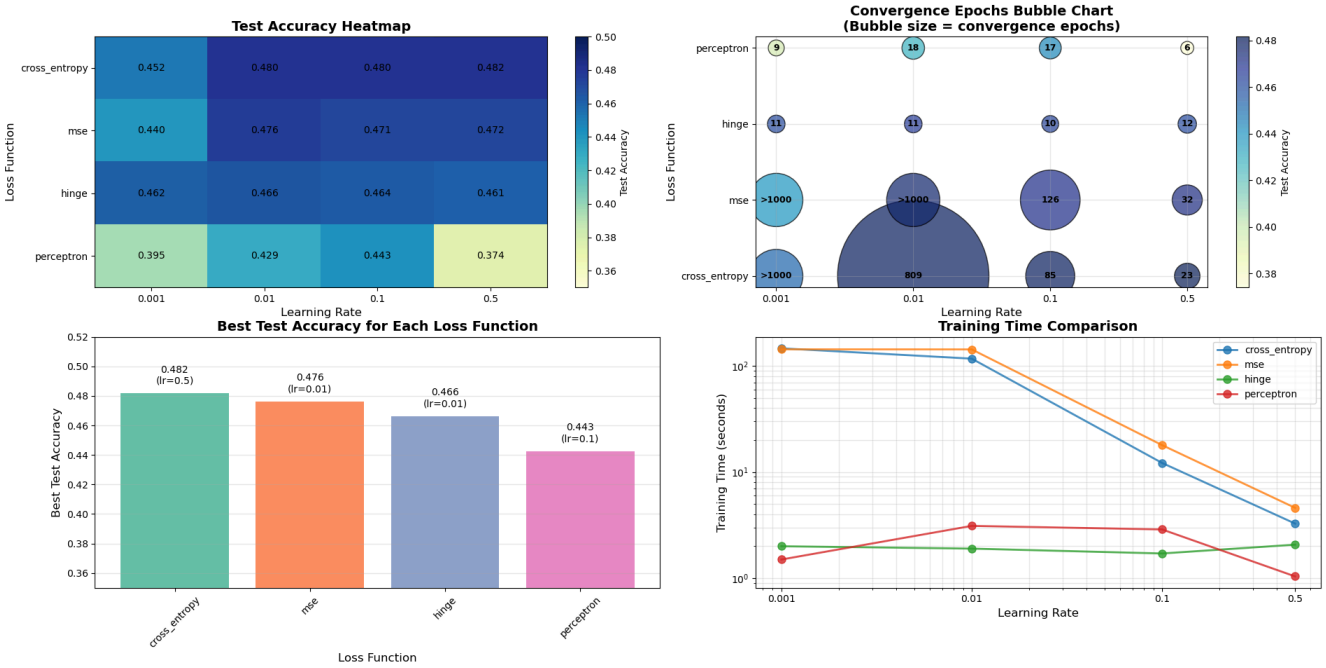
2. 实验二：对比不同损失函数和学习率

实验内容

- 损失函数选择范围：交叉熵损失函数（`Cross entropy`），均方损失函数（`MSE`）、`hinge` 损失函数（`hinge`）和感知器损失（`perceptron`）。
- `learning_rate` 选择范围：`[0.001, 0.01, 0.1, 0.5]`。

评价指标

- 测试集准确率
- 训练时间
- 早停时间



六、现象与分析

下面将对实验中出现的一些现象进行分析。

准确率随 `n` 的变化

现象：记 `n = (n_min, n_max)`，随着 `n_min` 的增大，整体的准确率呈现下降的趋势，当 `n_min` 固定时，`n_max` 取 `2` 时，整体的准确率最高。

分析：实验时发现，除了 `n=(1,1)` 的情况外，`n` 的其余取值进行向量化时，实际的特征数量都超过 `max_feature` 的限制。而在 `scikitlearn` 的向量化类中，`max_feature` 的实现方式是：首先找出所有的特征，保留出现频率最高的前 `max_feature` 个。

因此，当 `n_min` 固定时，提高 `n_max` 会产生更多长特征，这在一定程度上可以提供更丰富的上下文信息，例如 `good` 和 `not good`；而当 `n_max` 进一步增大，则可能引入了无用的噪声，并将更重要的短特征丢弃。而提高 `n_min` 时，可能有很多重要的短特征没有被学习，例如常用的 `good`，`bad` 等。

总体上来讲，随着 `n` 的增大，长度为 `n` 的特征出现的频率往往更低，因此当增大 `n_max` 时，新产生的特征频率往往较低，被保留的也就更少，因此对准确率的影响较小；而增大 `n_min` 时，原本多次出现的短特征被丢弃，向量化后的输入数据发生巨大变化，对准确率的影响较大。

因此，在该实验的情形下，`n=1` 的特征已经可以较好的表征一个句子的情感，`n=(1,2)` 则通过引入较高频的词组更好地辅助判断，而 `n_max` 进一步地提高则无法引入更好的特征。

不同损失函数对模型准确率、早停的影响

现象：交叉熵损失函数下，模型准确率最高，其次是均方误差损失函数和 `hinge` 损失函数，最低的是感知器损失函数。在同一学习率下，均方误差的迭代次数最多，其次是交叉熵损失函数，而 `hinge` 损失函数和感知器损失函数均迅速早停。

分析：

交叉熵损失函数和均方误差损失函数的梯度需要考虑样本对于不同标签的概率，即使预测概率接近 1，梯度依然不会消失，模型依然有微调的空间，因此较难发生早停。

而对于 `hinge` 损失函数和感知器损失，只要预测概率达到某个标准后，梯度就会消失，而由于采用 `n-gram` 构造特征，且样本量小，因此样本稀疏地分布在极高维空间中，可以轻易地找到一组超平面对样本进行线性分类，因此极快发生早停。